



Natural Language

- In the philosophy of language, a ***natural language*** is any language which arises in an unpremeditated fashion as the result of the innate facility for language possessed by the human intellect.
- A natural language is typically used for communication, and may be spoken, signed, or written.
- Natural language is distinguished from **formal languages** such as computer-programming languages.

Natural Language Processing

- Natural Language Processing (NLP) is the study of human languages and how they can be represented computationally and analyzed and generated algorithmically.
- NLP can also be defined as the study of building computational models of natural language comprehension and production.

Natural Language Processing

- Other Names of NLP:
 - ❑ Computational Linguistics (CL)
 - ❑ Human Language Technology (HLT)
 - ❑ Natural Language Engineering (NLE)
 - ❑ Speech and Text Processing
- Studying NLP involves studying natural language, formal representations, and algorithms for their manipulation.



Formal Language

- A ***formal language*** is a set of strings of symbols that may be constrained by rules that are specific to it.
- The ***alphabet*** of a formal language is the finite, nonempty set of symbols, letters, or tokens from which the strings of the language may be formed.
- The strings formed from this alphabet are called ***words***.
- A formal language is often defined by means of a **formal grammar**.



Formal Grammar

- A ***formal grammar*** is a set of production rules for strings in a formal language.
- The rules describe how to form strings from the language's alphabet that are valid according to the language's syntax.
- A grammar does not describe the meaning of the strings or what can be done with them in whatever context—only their form.



Informal Example of CFG's

Consider $L_{pal} = \{w \in \Sigma^* : w = w^R\}$

For example $otto \in L_{pal}$, $madamimadam \in L_{pal}$.

In Finnish language e.g. $saippuakauppias \in L_{pal}$
("soap-merchant")

Let $\Sigma = \{0, 1\}$ and suppose L_{pal} were regular.

Let n be given by the pumping lemma. Then $0^n 1 0^n \in L_{pal}$. In reading 0^n the FA must make a loop. Omit the loop; contradiction.

Let's define L_{pal} inductively:

Basis: ϵ , 0, and 1 are palindromes.

Induction: If w is a palindrome, so are $0w0$ and $1w1$.

Informal Example of CFG's



CFG's is a formal mechanism for definitions such as the one for L_{pal} .

1. $P \rightarrow \epsilon$
2. $P \rightarrow 0$
3. $P \rightarrow 1$
4. $P \rightarrow 0P0$
5. $P \rightarrow 1P1$

0 and 1 are *terminals*

P is a *variable* (or *nonterminal*, or *syntactic category*)

P is in this grammar also the *start symbol*.

1–5 are *productions* (or *rules*)



Formal Definition of CFG's

A *context-free grammar* is a quadruple

$$G = (V, T, P, S)$$

where

V is a finite set of *variables*.

T is a finite set of *terminals*.

P is a finite set of *productions* of the form $A \rightarrow \alpha$, where A is a variable and $\alpha \in (V \cup T)^*$

S is a designated variable called the *start symbol*.



Formal Definition Examples

Example: $G_{pal} = (\{P\}, \{0, 1\}, A, P)$, where $A = \{P \rightarrow \epsilon, P \rightarrow 0, P \rightarrow 1, P \rightarrow 0P0, P \rightarrow 1P1\}$.

Sometimes we group productions with the same head, e.g. $A = \{P \rightarrow \epsilon | 0 | 1 | 0P0 | 1P1\}$.

Example: Regular expressions over $\{0, 1\}$ can be defined by the grammar

$$G_{regex} = (\{E\}, \{0, 1\}, A, E)$$

where $A =$

$$\{E \rightarrow 0, E \rightarrow 1, E \rightarrow E.E, E \rightarrow E+E, E \rightarrow E^*, E \rightarrow (E)\}$$



Formal Definition Examples

Example: (simple) expressions in a typical prog lang. Operators are $+$ and $*$, and arguments are identifiers, i.e. strings in $L((a + b)(a + b + 0 + 1)^*)$

Solution:

The expressions are defined by the grammar

$$G = (\{E, I\}, T, P, E)$$

where $T = \{+, *, (,), a, b, 0, 1\}$ and P is the following set of productions:

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
5. $I \rightarrow a$
6. $I \rightarrow b$
7. $I \rightarrow Ia$
8. $I \rightarrow Ib$
9. $I \rightarrow I0$
10. $I \rightarrow I1$



Chomsky Hierarchy

- Grammatical formalisms can be classified by their ***generative capacity***: the set of languages they can represent.
- Noam Chomsky (1957) describes four classes of grammatical formalisms that differ only in the form of the rewrite rules.
- The classes can be arranged in a hierarchy, where each class can be used to describe all the languages that can be described by a less powerful class, as well as some additional languages.
- This hierarchy of grammars is known as ***Chomsky hierarchy***.



Chomsky Hierarchy

- Type-0 grammars (***recursively enumerable or unrestricted grammars***) include all formal grammars.
- They generate exactly all languages that can be recognized by a **Turing machine**.
- These languages are also known as the ***recursively enumerable languages***.



Chomsky Hierarchy

- Type-1 grammars (***context-sensitive grammars***) generate the ***context-sensitive languages***.
- These grammars have rules of the form $\alpha A \beta \rightarrow \alpha \gamma \beta$ with A a nonterminal and α , β and γ strings of terminals and nonterminals. The strings α and β may be empty, but γ must be nonempty. The rule $S \rightarrow \varepsilon$ is allowed if S does not appear on the right side of any rule.
- The languages described by these grammars are exactly all languages that can be recognized by a **linear-bounded Turing machine**.



Chomsky Hierarchy

- Type-2 grammars (***context-free grammars***) generate the ***context-free languages***.
- These are defined by rules of the form $A \rightarrow \gamma$ with A a nonterminal and γ a string of terminals and nonterminals.
- These languages are exactly all languages that can be recognized by a **pushdown automaton**.
- Context-free languages are the theoretical basis for the syntax of most programming languages.



Chomsky Hierarchy

- Type-3 grammars (***regular grammars***) generate the ***regular languages***.
- Such a grammar restricts its rules to a single nonterminal on the left-hand side and a right-hand side consisting of a single terminal, possibly followed (or preceded, but not both in the same grammar) by a single nonterminal.
- The rule $S \rightarrow \varepsilon$ is also allowed here if S does not appear on the right side of any rule.
- These languages are exactly all languages that can be decided by a **finite state automaton**. Additionally, this family of formal languages can be obtained by **regular expressions**.
- Regular languages are commonly used to define search patterns and the lexical structure of programming languages.



Chomsky Hierarchy

- The following table summarizes each of Chomsky's four types of grammars, the class of language it generates, the type of automaton that recognizes it, and the form its rules must have.



Chomsky Hierarchy

Grammar	Languages	Automaton	Production rules (constraints)
Type-0	Recursively enumerable	Turing machine	$\alpha \rightarrow \beta$ (no restrictions)
Type-1	Context-sensitive	Linear-bounded Turing machine	$\alpha A \beta \rightarrow \alpha \gamma \beta$
Type-2	Context-free	pushdown automaton	$A \rightarrow \gamma$
Type-3	Regular	Finite state automaton	$A \rightarrow a$ and $A \rightarrow aB$